Computer Graphics

Clipping & Culling

Back Face Culling

Dr. Hardik Jain
Assistant Professor
Department of Computer Science and Engineering
IIT Jodhpur

1

Motivation

Main problem

Many primitives either:

- Lie partially outside the view region → waste work if we rasterize them fully.
- Are fully hidden or outside the view → waste even more work.

Two families of optimizations

- **Clipping**: cut primitives against a region and discard the outside part.
- **Culling**: discard entire objects or primitives that cannot contribute to the final image.

2

Clipping

Geometric operation that restricts primitives to a specified region, discarding the parts outside.

3

Clipping

- Typical clip regions:
  - 3D: view frustum in clip space / NDC.
  - 2D: viewport / clip rectangle in window space.
- Why clip before rasterization?
  - Avoid rasterizing pixels that will never be shown.
  - Prevent numerical problems when projecting primitives that extend to infinity.
  - Enables later stages (rasterizer, depth test) to assume all fragments lie in a known finite region.
- Often, many points map to outside the range in the normalized 2D space.
- Think of the Frame buffer as an infinite canvas, of which a small rectangular portion is sent to the screen.
- So draw only the portion that is visible. That is, clip the primitives to a clip-rectangle.

4

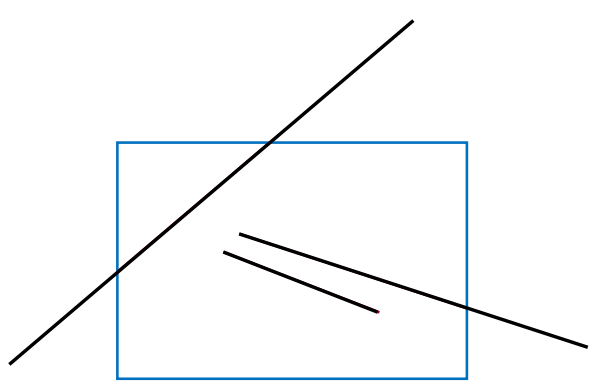
Clipping Points

- Clip rectangle defined by $[x_m, y_m]$ to $[x_M, y_M]$
- A point (x, y)
  - is inside if $x_m \leq x \leq x_M, y_m \leq y \leq y_M$
  - Else outside
- Can use this to clip any primitives
  - Simple, but perhaps we do more work than necessary.
- Analytically clip to the rectangle, then rasterize.
- **Scissoring**: Rasterize first, then test each pixel against the clip rectangle and skip writes outside. Could be easiest but inefficient.

5

Clipping Lines

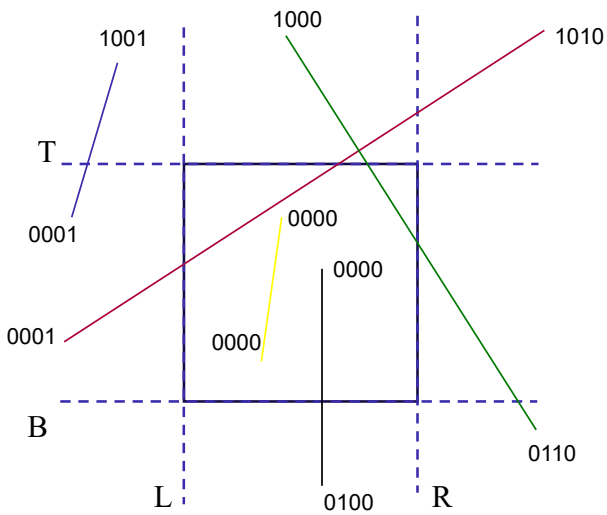
- World lines needn't map nicely onto points inside the window.
- End points map to window coordinates independently.
- **Clipping**: Finding part of the line that is inside the window or axis-aligned rectangle.



6

Cohen-Sutherland Algorithm

Clipping Lines

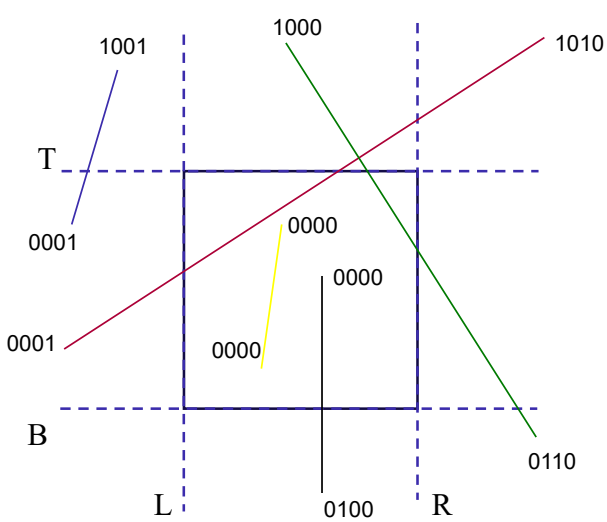


- Encode each end-point (vertex) of any line with binary region/out code
  - T B R L
  - Denoting 4 boundaries of the viewport
- $T = \begin{cases} 1 & \text{if a vertex is above the top edge T} \\ 0 & \text{else} \end{cases}$
- $B = \begin{cases} 1 & \text{if a vertex is below the bottom edge B} \\ 0 & \text{else} \end{cases}$
- $R = \begin{cases} 1 & \text{if a vertex is right to the right edge R} \\ 0 & \text{else} \end{cases}$
- $L = \begin{cases} 1 & \text{if a vertex is left to the left edge L} \\ 0 & \text{else} \end{cases}$

7

Cohen-Sutherland Algorithm

Clipping Lines



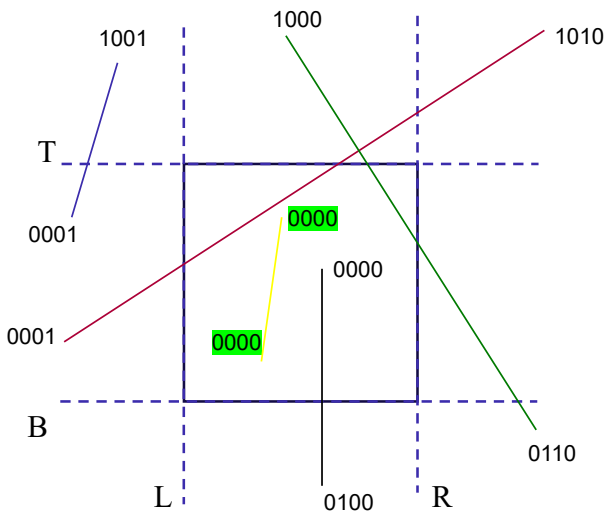
1. Line Inside the Window

- Take element-wise OR operation between the LRBT encoding of a pair of vertices for each line and if output is zero then the line is strictly inside the window.
- A vertex inside the window will always have LRBT coordinates as 0000

8

Cohen-Sutherland Algorithm

Clipping Lines



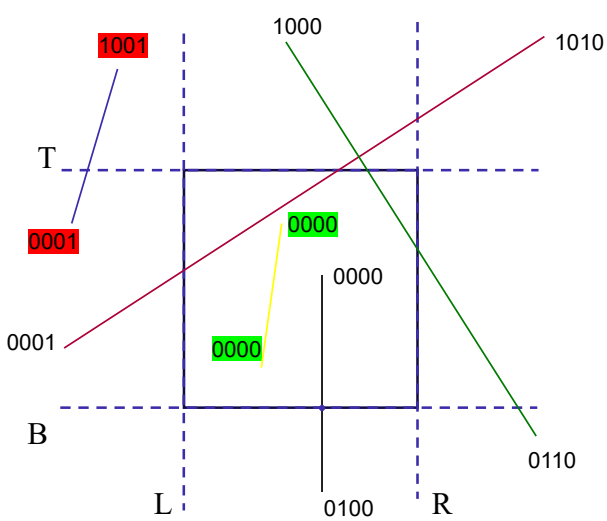
2. Line outside the Window

- Take element-wise AND & if the output is non-zero, then this line will never pass through viewport window.
- Eg:
  - Blue line:
    $\mathrm{AND}(1001,0001) = (0001)$
    Will not pass through viewport
  - Green Line
    $\mathrm{AND}(1000,0110) = (0000)$
    Could pass through viewport

9

Cohen-Sutherland Algorithm

Clipping Lines



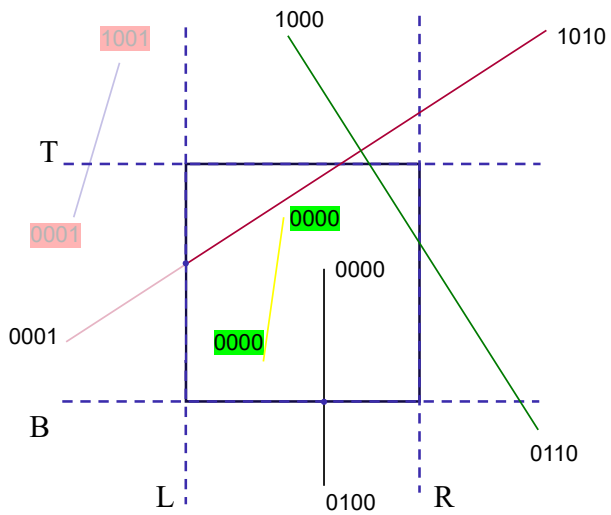
3. Line neither inside nor outside the Window

- Lines that cannot be identified as being completely inside or completely outside a clipping window by the region-code tests are next checked for intersection with the window border lines.
- Several intersection calculations might be necessary to clip a line segment, depending on the order in which we process the clipping boundaries.
- As each clipping-window edge is processed, a section of the line is clipped, and the remaining part of the line is checked against the other window borders.
- This is done until either the line is totally clipped or the remaining part of the line is inside the clipping window.

10

Cohen-Sutherland Algorithm

Clipping Lines



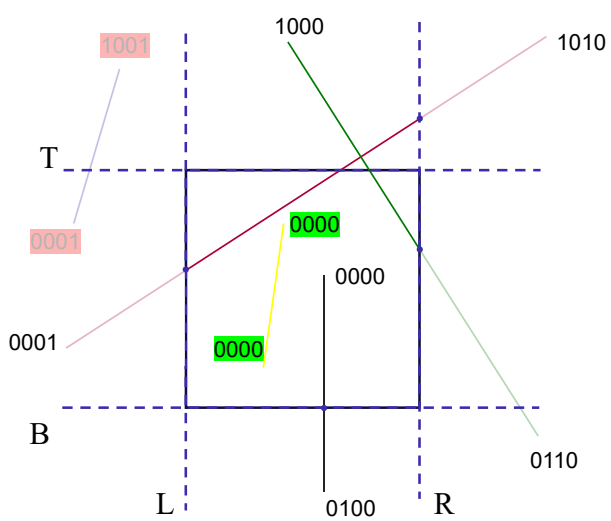
3. Intersecting Lines – Left

- Calculate the intersection position of line with the left borderline
- Clip off the line section until the left borderline
- Repeat for all lines which are not completely inside or outside

11

Cohen-Sutherland Algorithm

Clipping Lines



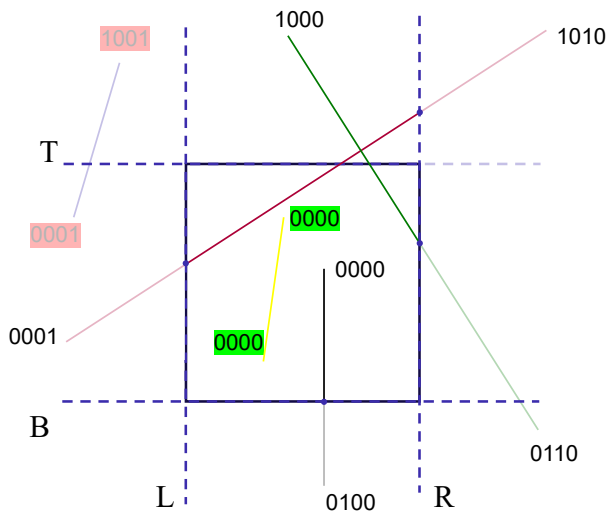
3. Intersecting Lines – Right

- Calculate the intersection position of line with the right borderline
- Clip off the line section until the right borderline
- Repeat for all lines which are not completely inside or outside

12

Cohen-Sutherland Algorithm

Clipping Lines



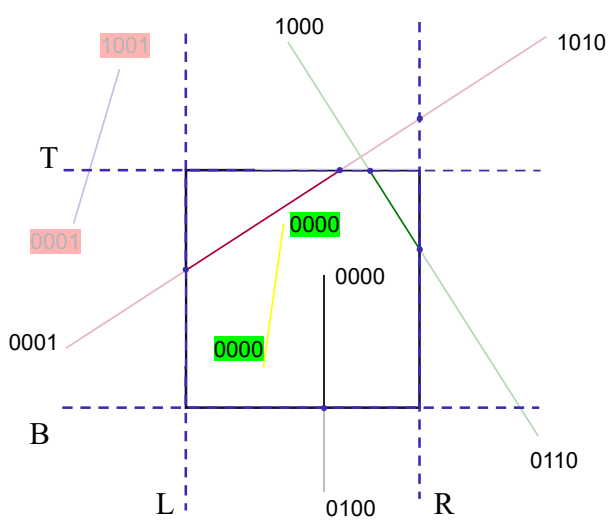
3. Intersecting Lines – Bottom

- Calculate the intersection position of line with the bottom borderline
- Clip off the line section until the bottom borderline
- Repeat for all lines which are not completely inside or outside

13

Cohen-Sutherland Algorithm

Clipping Lines



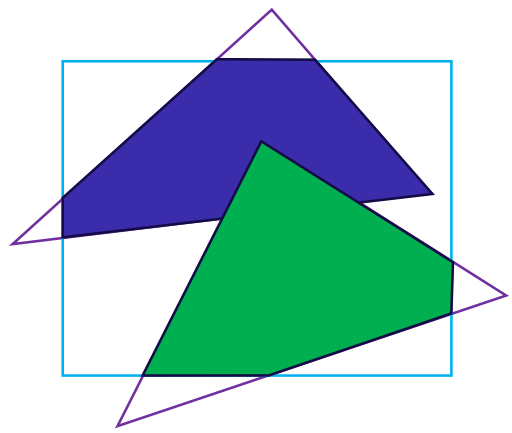
3. Intersecting Lines – Top

- Calculate the intersection position of line with the top borderline
- Clip off the line section until the top borderline
- Repeat for all lines which are not completely inside or outside

14

Clipping Triangles

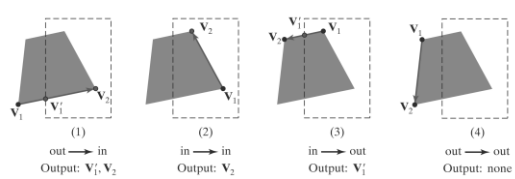
- Only parts of the triangle may lie in the window.
- First clip a triangle to a (planar!) polygon that lies inside.
- Scan convert the polygon subsequently.



Sutherland-Hodgman Algorithm

Process

- Treat the polygon as a list of vertices.
- For each clip edge in turn (Left, Right, Bottom, Top):
  - Walk all edges of the polygon (pairs of consecutive vertices).
  - Use four cases:
    - Inside → inside: output second vertex.
    - Outside → inside: output intersection point and second vertex.
    - Inside → outside: output intersection point only.
    - Outside → outside: output nothing.
  - The resulting vertex list becomes input to the next clip edge.

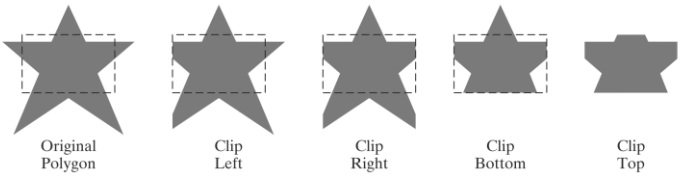


Sutherland-Hodgman Algorithm

- Restrict drawing/filling of a polygon to the inside of the clip rectangle.
- A convex polygon remains convex after clipping.
- A concave polygon may split into multiple polygons, but algorithm still works with a suitable extension.
- Can perform by intersecting to the four clip edges in turn.

Polygon Fill Area Clipping

- We can process a polygon fill area against the borders of a clipping window using the same general approach as in line clipping.
- A line segment is defined by its two endpoints, and these endpoints are processed through a line-clipping procedure by constructing a new set of clipped endpoints at each clipping-window boundary.
- Similarly, we need to maintain a fill area as an entity as it is processed through the clipping stages.
- Thus, we can clip a polygon fill area by determining the new shape for the polygon as each clipping-window edge is processed



Culling

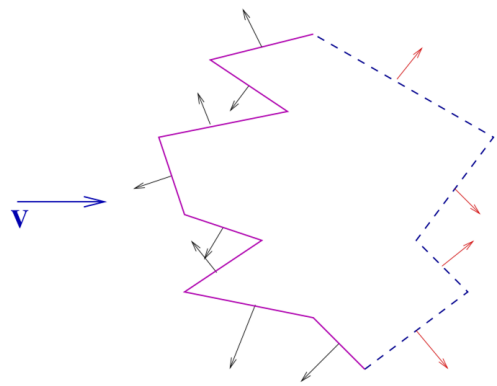
Discard entire objects or primitives that cannot contribute to the final image, without modifying their shape

Culling

- Culling is about visibility and performance, not about changing geometry.
- Typical levels:
  - **Object-level**: view frustum culling using bounding volumes.
  - **Primitive-level**: back-face culling for polygons.
  - **Pixel-level**: depth buffering (but this is usually treated separately under visibility/VSD).

Viewing a Solid Object

- Which lines are visible?
- Which are hidden?



Viewing a Solid Object: Back Face Culling

- Only the front side can be seen of solid, opaque objects.
- If surface normals point out of the object, polygons with normals pointing away from the viewer cannot be seen.
- Let v be the vector from COP to any point on the polygon and n be the surface normal.
- If $v \cdot n < 0$, draw. Else discard.
- After projection normalizing, DOP is $[0\ 0\ -1]^{\mathrm{T}}$. Eliminate polygons with negative z component in the normal vector. Because if the polygon faces the camera, its normal will have negative z in this coordinate system.
- Half the polygons eliminated on the average.

Comparison

Clipping	Culling
Modifies primitives so only the inside part remains; mandatory for correctness at the frustum/viewport.	Throws away whole objects/primitives to avoid unnecessary work; optional (an optimization

First reduce whole objects (culling), then cut remaining primitives to the visible region (clipping), then rasterize and resolve final visibility per pixel.

Thank You

Next Stage

Rasterization